# Design-First Collaboration

*AI coding assistants default to generating implementation immediately — embedding design decisions invisibly in the output. I propose a structured conversation pattern that mirrors whiteboarding with a human pair: progressive levels of design alignment before any code, reducing cognitive load and catching misunderstandings at the cheapest possible moment.*

03 March 2026

Rahul Garg

Rahul is a Principal Engineer at Thoughtworks, based in Gurgaon, India. He is passionate about the craft of building maintainable software through DDD and Clean Architecture, and explores how AI can help teams achieve engineering excellence.

CONTENTS



SIDEBARS



GENERATIVE AI

This article is part of a series:
**Patterns for Reducing Friction in AI-Assisted Development**

When I pair program with a colleague on something complex, we don't start at the keyboard. We go to the whiteboard. We sketch components, debate data f

about boundaries. We align on what the system needs to do before discussing how to build it. Only after this alignment — sometimes quick, sometimes extended — do we sit down and write code. The whiteboarding is not overhead. It is where the real thinking happens, and it is what makes the subsequent code right. The principle is simple: *whiteboard before keyboard*.

With AI coding assistants, this principle vanishes entirely. The speed is seductive: describe a feature, receive hundreds of lines of implementation in seconds. The AI may understand the requirement perfectly well — an email notification service with retry logic, say. But understanding *what* to build and collaborating on *how* to build it are two different activities, and AI collapses them into one. It does not pause to discuss which components to create, whether to use existing infrastructure or introduce new abstractions, what the interfaces should look like. It jumps from requirement to implementation, making every technical design decision silently along the way.

I have come to think of this as the "Implementation Trap." The AI produces tangible output so quickly that the natural checkpoint between *thinking about design* and *writing code* disappears. The result is not just misaligned code. It is the cognitive burden of untangling design decisions I was never consulted on, bundled inside an implementation I now have to review line by line.

## The Cost of Invisible Design

The Implementation Trap is not simply that AI skips design. In a meaningful sense, the AI *does* make design decisions when it generates code — about scope, component boundaries, data flow, interfaces, error handling. But those decisions arrive silently, embedded in the implementation. There is no moment where I can say "wait, we already have a queue system" or "that interface won't work with our existing services." The first time I see the AI's design thinking is when I am reading code, which is the most expensive and cognitively demanding place to discover a disagreement.

This, I believe, is why reviewing AI-generated code feels so much more exhausting than reviewing a colleague's work. When a human pair submits code after a whiteboarding session, I am reviewing implementation against a design I already understand and agreed to. When AI generates code from a single prompt, I am simultaneously evaluating scope (did it build what I needed?), architecture (are the component boundaries right?), integration (does it fit our existing infrastructure?), contracts (are the interfaces correct?), and code quality (is the implementation clean?) — all at once, all entangled.

That is too many dimensions of judgment for a single pass. The brain is nc 

Things get missed — not because I am careless, but because I am overloaded.

Software engineers have understood this principle since the 1980s. Barry Boehm's Cost of Change Curve demonstrated that fixing a requirements misunderstanding at the design stage costs a fraction of fixing it in implementation — and orders of magnitude less than fixing it in production. The same economics apply to AI collaboration: catching a scope mismatch in a two-minute design conversation is fundamentally cheaper than discovering it woven through 400 lines of generated code.

There is an additional cost that is easy to overlook. AI tends to add features. A request for a notification service might return code with rate limiting, analytics hooks, and a webhook system — none of which were requested. This is not just scope creep; it is what I would call *technical debt injection*. Every unrequested addition is code I did not ask for but must review, tests I did not plan but must write, surface area I did not need but must maintain. And because it arrives looking clean and reasonable, it is easy to accept without questioning whether it belongs. The cognitive load of the review process increases, because I am now evaluating code that solves problems I never asked about.

## Reconstructing the Whiteboard

The solution, I believe, is to reconstruct the whiteboarding conversation that human pairs do naturally — making the AI's implicit design thinking explicit and collaborative. Rather than asking for implementation directly, I walk through progressive levels of design. Each level surfaces a category of decisions that would otherwise be buried in generated code.

I use five levels, ordered from abstract to concrete:

1. Capabilities — What does this system need to do? Core requirements only, no implementation detail.
2. Components — What are the building blocks? Services, modules, major abstractions.
3. Interactions — How do the components communicate? Data flow, API calls, events.
4. Contracts — What are the interfaces? Function signatures, types, schemas.
5. Implementation — Now write the code.

The key constraint: *no code until Level 5 is approved.*

This single rule changes the dynamics of the collaboration. Each level becomes a checkpoint — a place where I can agree, disagree, or redirect before any code exists.

Level 1 serves as a shared vocabulary check — confirming that the AI and I are talking about the same feature, with the same scope, before any design begins. The deeper value starts at Level 2, where the real technical design conversation unfolds: component boundaries, then interaction patterns, then contracts. At each level, I am thinking about one category of decision, not all of them at once. This is cognitive load management — the brain is never juggling everything at once. And because both human and AI align at each step, a shared mental model builds incrementally, just as it does at a whiteboard.

This is how effective human pairs work. At a whiteboard, a colleague does not propose an entire implementation in one breath. They sketch a box, explain what it does, wait for feedback, then sketch the next box. The shared mental model builds incrementally. Design-First applies the same discipline to AI collaboration: a sequential conversation where alignment builds step by step, and where each step constrains the decision space for the next.

Without this structure, the AI is not really a pair — it is a colleague who went off to a separate room, made all the decisions alone, and returned with finished code. Design-First makes the collaboration real. Both human and AI examine each layer of the design together, and by the time implementation begins, there is genuine shared understanding of what is being built and why.

## What This Looks Like in Practice

Recently, while building a notification service, this approach proved its worth almost immediately.

I came to the conversation with clear context: the feature scope from our story — email-only delivery for v1, with retry logic and basic status tracking — and the project's tech stack, including BullMQ for job processing. At the Capabilities level, I shared this scope upfront and asked the AI to confirm its understanding before any design began. A quick check, but an important one: it ensured we were speaking the same language about what was being built, and what was explicitly out of scope.

The real design conversation began at the Components level. The AI proposed five components, including a dedicated `RetryQueue` abstraction wrapping BullMQ. Architecturally clean on paper — but unnecessary. BullMQ's built-in retry mechanism with exponential backoff handles this natively, and an additional abstraction would add indirection without value. I pushed back: use BullMQ directly, no wrapper. The AI simplified the component structure. This was not a context gap — the AI knew about BullMQ from the project context I had shared. It was a genuine design dis

about whether to abstract over existing infrastructure, resolved in seconds because no code existed yet.

At the Interactions level, this simpler component structure had a cascading benefit. Without the unnecessary abstraction layer, the data flow was more direct: the handler queues a BullMQ job, the worker processes it, retries are handled natively. The interaction design was cleaner and better integrated than anything that included the wrapper would have been.

The Contracts level is where something particularly valuable happens. Once I had agreed-upon function signatures, types, and interfaces — `NotificationPayload`, `NotificationResult`, `EmailProvider`, `DeliveryTracker` — I had everything I needed to ask the AI to write tests *before* any implementation code. The design conversation had, almost as a side effect, created the preconditions for test-driven development. Approve contracts, generate tests, then implement against those tests. For teams that practice TDD, Design-First provides a natural on-ramp. For teams that do not, the contracts still serve as a safety net — a concrete, reviewable specification that makes misunderstandings visible before they become bugs.

By the time implementation began at Level 5, the code was far more aligned than anything a single prompt would have produced. Scope was confirmed. Architecture fit the existing infrastructure — not because the AI was told about it mid-conversation, but because we debated how to use it at the design level. Contracts were defined and tested. The AI was not guessing — it was implementing an agreed design.

## Discipline and Calibration

This approach requires a form of discipline that cuts against how AI assistants are typically used. AI is trained to be helpful, and "helpful" usually means producing tangible output quickly. Left unchecked, it will jump to implementation. It will combine levels, offering component diagrams with code already written, or proposing contracts with implementations attached. The discipline of saying "stop — we are still at Level 2, show me only the component structure" is really about protecting working memory from premature detail. It keeps the conversation at the right level of abstraction for the decision being made.

Not every task needs all five levels. The framework scales to the complexity of the work:

| Task Complexity | Start At | Example |
| --- | --- | --- |

| Task Complexity | Start At | Example |
|---|---|---|
| Simple utility | Level 4 (Contracts) | Date formatter, string helper |
| Single component | Level 2 (Components) | Validation service, API endpoint |
| Multi-component feature | Level 1 (Capabilities) | Notification system, payment integration |
| New system integration | Level 1 + deep Level 3 | Third-party API, event-driven pipeline |

The framework is a tool for managing complexity, not a ritual to be applied uniformly.

Design-First also compounds with what I have called Knowledge Priming — sharing curated project context (tech stack, conventions, architecture decisions) with the AI before beginning work. When the AI already understands the project's architecture, naming conventions, and version constraints, each design level is already anchored to the codebase. The Capabilities discussion reflects real requirements. The Components level uses existing naming patterns. The Contracts match established type conventions. Priming provides the vocabulary; Design-First provides the structure. Together, they create a conversation where alignment happens naturally rather than through constant correction.

In practice, a single opening prompt is often enough to set the entire pattern in motion:

```
I need to build [feature]. Before writing any code, walk me through the design:

- Capabilities: [your scope constraints]
- Components: [your infrastructure and architecture constraints]
- Interactions: [your integration and data flow patterns]
- Contracts: [your type and interface conventions]

Present each level separately. Wait for my approval before moving to the next.
No code until the contracts are agreed.
```

This prompt sets the sequential discipline — the AI will present each level, wait for feedback, and hold off on code until given explicit approval. But the real value is in the brackets: the project-specific constraints each developer fills in. “Email only for v1.” “Use existing BullMQ infrastructure.” “Functional patterns, no classes.” “All interfaces must align with our established type conventions.” These constraints carry forward through every level, shaping each design decision without needing to be repeated. At each checkpoint, the conversation deepens — I add context, correct assumptions, steer

the design based on what I know about the codebase. The prompt starts the motion; the collaboration at each level shapes it.

Over time, teams that find this pattern valuable often encode it as a reusable, shareable instruction — enriching it with architectural guardrails, quality expectations, and team-specific defaults that apply automatically to every design conversation.

This approach is not without costs. Design conversations take longer than immediately requesting code. For trivial tasks, the overhead is not justified. There is also a learning curve — developers accustomed to typing a prompt and receiving code may find the sequential structure unfamiliar at first. The discipline to sustain it requires conscious effort, especially when deadlines press and the temptation to "just ask for the code" is strong.

I believe the investment pays off primarily for non-trivial work — the kind where a misunderstanding costs not minutes but hours, where architectural alignment matters, where the code will be maintained and extended long after the initial conversation ends.

**This pattern in Lattice**

The `design-first` atom in Lattice encodes the five-level methodology as an installable skill. The `design-blueprint` molecule orchestrates a full design workflow — loading context, walking the levels, persisting the approved blueprint — so the practice survives deadline pressure without relying on individual discipline.

## Conclusion

The value of the whiteboard was never the diagram. It was the alignment — the shared understanding that develops when two people think through a problem together, one layer at a time. Design-First applies this principle to AI collaboration.

When both human and AI operate from the same scope, the same component architecture, the same interaction model, the same contracts, the cognitive load shifts from exhausted vigilance to collaborative refinement. The developer's role changes from reverse-engineering invisible design decisions to steering them explicitly. This, I believe, is what it means to actually pair with an AI — not just receive its output, but participate in its thinking.

The simplest version of this entire approach is a single constraint: no code until the design is agreed. Everything else follows from there.

**Table of Contents**

## ▸ Significant Revisions